

# プログラミング言語屋から見た医療

Haochen Kotoi-Xie

[github.com/haochenx](https://github.com/haochenx)

所属: KXC  
(Kotoi-Xie Consultancy, Inc.)

東海医療 DX 研究会 #7

05SEP 2026

「プログラム」と聞いて、  
どんな姿を思い浮かべますか？

```
if (t >= 37.5) { ...}  SELECT ... FROM ...   $\lambda x. \lambda y. \dots$   10 PRINT ...  →→→
```

# たとえば、こんな？

①

```
#include <stdio.h>
int main(void) {
    double t = 37.8;
    if (t >= 37.5)
        puts("fever");
    return 0;
}
```

④

```
(define (mean xs)
  (/ (apply + xs)
     (length xs)))
(mean '(36.2 37.8 36.5))
```

②

```
10 INPUT "T="; T
20 IF T >= 37.5 THEN 50
30 PRINT "OK"
40 GOTO 10
50 PRINT "FEVER"
60 GOTO 10
```

⑤

```
let mean xs =
  List.fold_left
    (+.) 0. xs
  /. float
    (List.length xs)
```

③

```
program vitals
  real :: t(3)
  t = (/36.2,37.8,36.5/)
  print *, 'mean =', &
    sum(t) / size(t)
end program
```

⑥

```
SELECT name, visited_on
FROM karte
WHERE icd10 LIKE 'E11%'
ORDER BY visited_on;
```

# たとえば、こんな？

## ① C

```
#include <stdio.h>
int main(void) {
    double t = 37.8;
    if (t >= 37.5)
        puts("fever");
    return 0;
}
```

## ④ Scheme

```
(define (mean xs)
  (/ (apply + xs)
     (length xs)))
(mean '(36.2 37.8 36.5))
```

## ② BASIC

```
10 INPUT "T="; T
20 IF T >= 37.5 THEN 50
30 PRINT "OK"
40 GOTO 10
50 PRINT "FEVER"
60 GOTO 10
```

## ⑤ OCaml

```
let mean xs =
  List.fold_left
    (+.) 0. xs
  /. float
    (List.length xs)
```

## ③ Fortran

```
program vitals
  real :: t(3)
  t = (/36.2,37.8,36.5/)
  print *, 'mean =', &
    sum(t) / size(t)
end program
```

## ⑥ SQL

```
SELECT name, visited_on
FROM karte
WHERE icd10 LIKE 'E11%'
ORDER BY visited_on;
```

# それとも、こんな？

⑦

スクリプト：患者カルテ検索

検索モードに切り替え []

フィールド設定

[患者::患者ID; Get(スクリプト引数)]

検索実行 []

If [Get(対象レコード数) = 0]

カスタムダイアログを表示

["該当患者なし"]

End If

レイアウト切り替え [「カルテ一覧」]

⑧

Sub 体温チェック()

Dim r As Range

For Each r In Range("C2:C40")

If r.Value >= 37.5 Then

r.Interior.Color = vbRed

r.Offset(0, 1).Value = "発熱"

End If

Next r

End Sub

# それとも、こんな？

## ⑦ FileMaker (電子カルテ)

スクリプト：患者カルテ検索

検索モードに切り替え []

フィールド設定

[患者::患者ID; Get(スクリプト引数)]

検索実行 []

If [Get(対象レコード数) = 0]

カスタムダイアログを表示

["該当患者なし"]

End If

レイアウト切り替え [「カルテ一覧」]

## ⑧ VBA (Excel マクロ)

Sub 体温チェック()

Dim r As Range

For Each r In Range("C2:C40")

If r.Value >= 37.5 Then

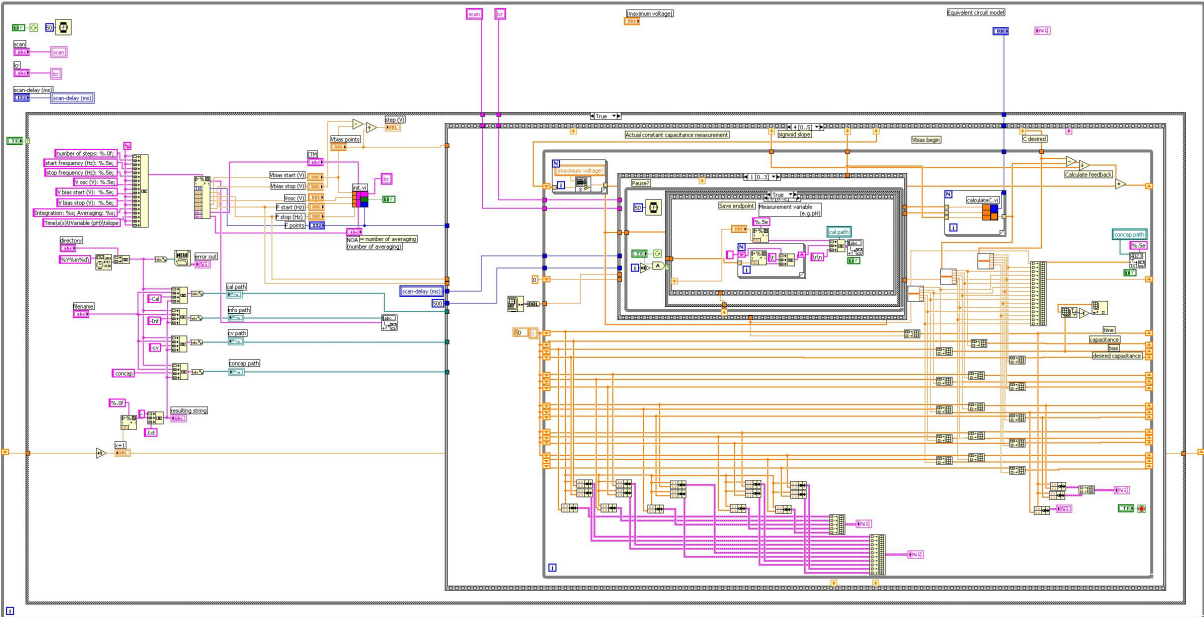
r.Interior.Color = vbRed

r.Offset(0, 1).Value = "発熱"

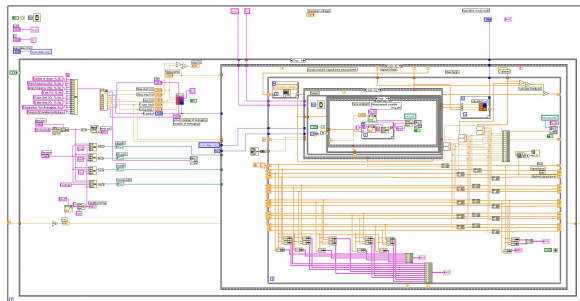
End If

Next r

End Sub



# これは LabVIEW



## LabVIEW

National Instruments, 1986～

計測・制御分野の定番

言語の名前は “G”

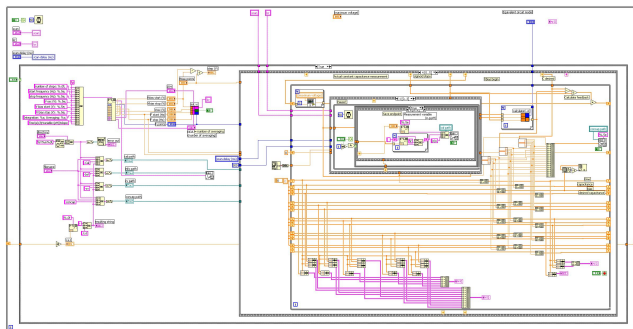
グラフィカルなデータフロー言語

プログラム = 「Virtual Instrument」

(拡張子 = .vi)

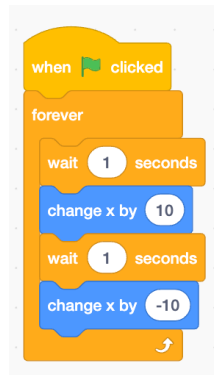


# グラフィカル言語つながりで



LabVIEW (G 言語)

プログラム = データフローダイアグラム  
としての「Virtual Instrument」



Scratch

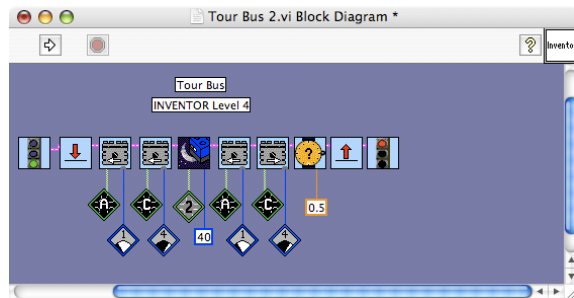
プログラム = 各スレッドのコードを  
ブロック構文で表現したもの

# プログラミングとの出会い

発表者が小 3 のとき：LEGO Mindstorms を **ROBOLAB** でプログラミング



RCX 1.0



ROBOLAB — LabVIEW 製 (ウィンドウ名も .vi)



# どの言語も、だいたい同じ？

よく似た言語たち

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

# どの言語も、だいたい同じ？

よく似た言語たち

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

$$\begin{aligned}
 &(\lambda f.(\lambda x.f (x x)) (\lambda x.f (x x))) \quad \lambda k.\lambda s.k (s s) \\
 &(\lambda x.\lambda y.\lambda z.x z (y z)) (\lambda u.u) \quad \lambda n.\lambda f.\lambda x.f (n f x) \\
 &(\lambda g.g g) (\lambda g.\lambda x.g g x)
 \end{aligned}$$

# どの言語も、だいたい同じ？

よく似た言語たち

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

$$\begin{aligned}
 &(\lambda f. (\lambda x. f \ (x \ x)) \ (\lambda x. f \ (x \ x))) \ \lambda k. \lambda s. k \ (s \ s) \\
 &(\lambda x. \lambda y. \lambda z. x \ z \ (y \ z)) \ (\lambda u. u) \ \lambda n. \lambda f. \lambda x. f \ (n \ f \ x) \\
 &(\lambda g. g \ g) \ (\lambda g. \lambda x. g \ g \ x)
 \end{aligned}$$

これもプログラム ( $\lambda$  記法)

# どの言語も、だいたい同じ？

よく似た言語たち

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

関数型

OCaml

Haskell

Clojure

Lisp (Common Lisp / Scheme)

# どの言語も、だいたい同じ？

## よく似た言語たち

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

## 関数型

OCaml

Haskell

Clojure

Lisp (Common Lisp / Scheme)

## 論理型

Prolog

Datalog

## データフロー

LabVIEW

Simulink

Excel (スプレッドシート)

## 関係型

miniKanren

SQL\*

\*かなり強引に入れれば

# どの言語も、だいたい同じ？

## よく似た言語たち

命令型を主とした

## マルチパラダイム

## 関数型

## 論理型

## データフロー

## 関係型

※パラダイムはここに挙げたものだけではない

Java

JavaScript

C#

Swift

PHP

Python

VB

Kotlin

OCaml

Haskell

Clojure

Lisp (Common Lisp / Scheme)

Prolog

Datalog

LabVIEW

Simulink

Excel (スプレッドシート)

miniKanren

SQL\*

\*かなり強引に入れれば



# プログラミング言語とは？

一般的には：

計算機に実行してほしい計算の「内容」や「方法」を指定するための、  
データとして表現可能な（人工）言語

# プログラミング言語とは？

一般的には：

計算機に実行してほしい計算の「内容」や「方法」を指定するための、  
データとして表現可能な（人工）言語

なぜ「内容や方法」？ ← 大別すると二種類ある

# プログラミング言語とは？

一般的には：

計算機に実行してほしい計算の「内容」や「方法」を指定するための、  
データとして表現可能な（人工）言語

なぜ「内容や方法」？ ← 大別すると二種類ある

計算の手順を指定する



命令型言語

C ・ Java ・ BASIC ・ ...

計算したいものの性質を指定する



宣言型言語

SQL ・ Prolog ・ miniKanren ・ ...

# プログラミング言語とは？

一般的には：

計算機に実行してほしい計算の「内容」や「方法」を指定するための、  
データとして表現可能な（人工）言語

なぜ「内容や方法」？ ← 大別すると二種類ある

計算の**手順**を指定する



命令型言語

C ・ Java ・ BASIC ・ ...

計算したいものの**性質**を指定する



宣言型言語

SQL ・ Prolog ・ miniKanren ・ ...

ただし純粋にどちらかではなく、混ざっているケースも多い

# プログラミング言語とは？

より広義には、こういうものも含む：

振舞いを規定する形式言語  
情報処理システムの振舞いを  
規定しうる言語

データ言語

XML

HTML

JSON

YAML

その他

CSS

正規表現

BNF

など

理論上の言語

計算や計算機を系統的に理解する  
ために作られた言語

$(\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x)))$

← さっきの「 $\lambda$  記法」(ラムダ計算) もこれ

# プログラミング言語論とは？

# プログラミング言語論とは？

**NOT**

~~プログラミング言語サムライ~~

# プログラミング言語論とは？

**NOT**

~~プログラミング言語ソムリエ~~

実際に研究しているのは：

意味論

型システム

処理系

プログラム検証

抽象化手法

など



# プログラミング言語論とは？

NOT

~~プログラミング言語ソムリエ~~

実際に研究しているのは：

意味論

型システム

処理系

プログラム検証

抽象化手法

など

プログラミング言語論 = Programming Language Theory (ふだんは略して「PL」)

ここで、言語をひとつ覚えてもらいます。

# miniKanren

関係プログラミング (Relational Programming) の代表格

今日の本題に深く関わるので、ぜひ身につけて帰ってください

# 関係で書けると、何が起きる？

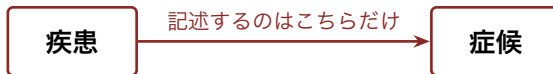
関係プログラミングでは、対象と対象の「関係」を記述する。

疾患

症候

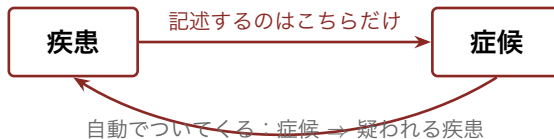
# 関係で書けると、何が起きる？

関係プログラミングでは、対象と対象の「関係」を記述する。



# 関係で書けると、何が起きる？

関係プログラミングでは、対象と対象の「関係」を記述する。



# 関係で書けると、何が起きる？

関係プログラミングでは、対象と対象の「関係」を記述する。



絞り込みも書ける： 発熱・咳 ⇒ インフルエンザ・肺炎・麻疹

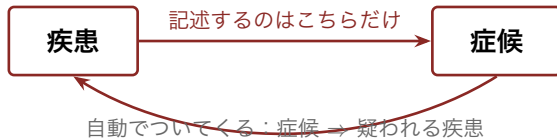
+発疹なし ⇒ インフルエンザ・肺炎（麻疹は除外）

= 計算機に支援される鑑別診断

※概念説明のための模式的な例です

# 関係で書けると、何が起きる？

関係プログラミングでは、対象と対象の「関係」を記述する。



絞り込みも書ける： 発熱・咳 ⇒ インフルエンザ・肺炎・麻疹

+発疹なし ⇒ インフルエンザ・肺炎（麻疹は除外）

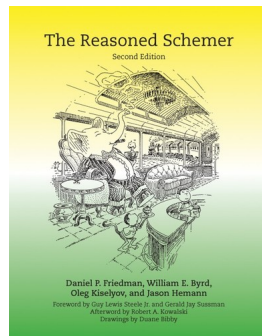
さらに——「残った候補を鑑別する症候はどれ？」とも問える  
＝計算機が導く鑑別診断

※概念説明のための模式的な例です

# miniKanren — 関係プログラミング



William E. Byrd (Will)  
miniKanren の生みの親のひとり



*The Reasoned Schemer*, 2nd ed.  
(MIT Press)  
「教科書」はこちら



# 関数と関係 — 何がちがう？

## 関数

入力  $\xrightarrow{f}$  出力

「計算する」 （向きは一方通行）

## 関係

$R(x, y)$  —  $x$  と  $y$  のあいだの制約

「成り立つ」 （どの向きにも問える）

関数に聞けること:  $f(x)$  は何？

関係に聞けること:  $y$  は何？  $x$  は何？ 両方未知でもいい

# 道具は 3 つだけ

`run* / ==`

```
(run* (q)
  (== q 'fever))
```

$\Rightarrow$  (fever)

問い合わせ+等値制約

`fresh`

```
(run* (q)
  (fresh (x)
    (== x 37.8)
    (== q x)))
```

$\Rightarrow$  (37.8)

論理変数を導入する

`conde`

```
(run* (q)
  (conde
    ((== q 'yes))
    ((== q 'no))))
```

$\Rightarrow$  (yes no)

選言条件 (1 つ以上が成立)

この 3 つの文法だけで関係プログラミングが可能。「代入」と「return」は使わない。

※miniKanren は Scheme のようなホスト言語に埋め込んで使う——この 3 つは、その「追加分」の Primitives

# 道具は 3 つだけ

```
run* / ==
```

```
(run* (q)  
  (== q 'fever))
```

```
⇒ (fever)
```

問い合わせ + 等値制約

```
fresh
```

```
(run* (q)  
  (fresh (x)  
    (== x 37.8)  
    (== q x)))
```

```
⇒ (37.8)
```

論理変数を導入する

```
conde
```

```
(run* (q)  
  (conde  
    ((== q 'yes))  
    ((== q 'no))))
```

```
⇒ (yes no)
```

選言条件 (1 つ以上が成立)

この 3 つの文法だけで関係プログラミングが可能。「代入」と「return」は使わない。

※miniKanren は Scheme のようなホスト言語に埋め込んで使う——この 3 つは、その「追加分」の Primitives

# 道具は 3 つだけ

`run* / ==`

```
(run* (q)
  (== q 'fever))
```

⇒ (fever)

問い合わせ + 等値制約

`fresh`

```
(run* (q)
  (fresh (x)
    (== x 37.8)
    (== q x)))
```

⇒ (37.8)

論理変数を導入する

`conde`

```
(run* (q)
  (conde
    ((== q 'yes))
    ((== q 'no))))
```

⇒ (yes no)

選言条件 (1 つ以上が成立)

この 3 つの文法だけで関係プログラミングが可能。「代入」と「return」は使わない。

※miniKanren は Scheme のようなホスト言語に埋め込んで使う——この 3 つは、その「追加分」の Primitives

# 道具は 3 つだけ

`run* / ==`

```
(run* (q)
  (== q 'fever))
```

⇒ (fever)

問い合わせ + 等値制約

`fresh`

```
(run* (q)
  (fresh (x)
    (== x 37.8)
    (== q x)))
```

⇒ (37.8)

論理変数を導入する

`conde`

```
(run* (q)
  (conde
    ((== q 'yes))
    ((== q 'no))))
```

⇒ (yes no)

選言条件 (1 つ以上が成立)

この 3 つの文法だけで関係プログラミングが可能。「代入」と「return」は使わない。

※miniKanren は Scheme のようなホスト言語に埋め込んで使う——この 3 つは、その「追加分」の Primitives

# はじめての関係 — 症状と診療科

**定義（関係名は -o で終える約束）**

```
(define (shinryoka-o sympt ka)
  (conde
    ((= sympt '発熱) (= ka '内科))
    ((= sympt '頭痛) (= ka '内科))
    ((= sympt '頭痛)
     (= ka '脳神経外科))
    ((= sympt '骨折)
     (= ka '整形外科))))
```

# はじめての関係 — 症状と診療科

## 定義（関係名は -o で終える約束）

```
(define (shinryoka-o sympt ka)
  (conde
    ((= sympt '発熱) (= ka '内科))
    ((= sympt '頭痛) (= ka '内科))
    ((= sympt '頭痛)
     (= ka '脳神経外科))
    ((= sympt '骨折)
     (= ka '整形外科))))
```

## ふつうの向きでクエリ

```
(run* (ka)
  (shinryoka-o '頭痛 ka))
⇒ (内科 脳神経外科)
```

# はじめての関係 — 症状と診療科

## 定義（関係名は -o で終える約束）

```
(define (shinryoka-o sympt ka)
  (conde
    ((= sympt '発熱) (= ka '内科))
    ((= sympt '頭痛) (= ka '内科))
    ((= sympt '頭痛)
     (= ka '脳神経外科))
    ((= sympt '骨折)
     (= ka '整形外科))))
```

## ふつうの向きでクエリ

```
(run* (ka)
      (shinryoka-o '頭痛 ka))
```

⇒ (内科 脳神経外科)

## 逆向きでクエリ

```
(run* (sympt)
      (shinryoka-o sympt '内科))
```

⇒ (発熱 頭痛)

ひとつの定義、どちら向きにも問える。



# 関係プログラミングの典型例 — appendo (リスト連結の「関係」)

## 定義

```
(define (appendo l r out)
  (conde
    ((== l '()) (== r out))
    ((fresh (a d res)
      (== `(,a . ,d) l)
      (== `(,a . ,res) out)
      (appendo d r res))))))
```

# 関係プログラミングの典型例 — appendo (リスト連結の「関係」)

## 定義

```
(define (appendo l r out)
  (conde
    ((= l '()) (= r out))
    ((fresh (a d res)
      (= `(,a . ,d) l)
      (= `(,a . ,res) out)
      (appendo d r res))))))
```

## 連結

```
(run* (q) (appendo '(a b) '(c) q))
⇒ ((a b c))
```

# 関係プログラミングの典型例 — appendo (リスト連結の「関係」)

## 定義

```
(define (appendo l r out)
  (conde
    ((= l '()) (= r out))
    ((fresh (a d res)
      (= `(,a . ,d) l)
      (= `(,a . ,res) out)
      (appendo d r res))))))
```

## 連結

```
(run* (q) (appendo '(a b) '(c) q))
⇒ ((a b c))
```

## 逆算

```
(run* (q) (appendo q '(c) '(a b c)))
⇒ ((a b))
```

# 関係プログラミングの典型例 — appendo (リスト連結の「関係」)

## 定義

```
(define (appendo l r out)
  (conde
    ((= l '()) (= r out))
    ((fresh (a d res)
      (= `(,a . ,d) l)
      (= `(,a . ,res) out)
      (appendo d r res))))))
```

## 連結

```
(run* (q) (appendo '(a b) '(c) q))
⇒ ((a b c))
```

## 逆算

```
(run* (q) (appendo q '(c) '(a b c)))
⇒ ((a b))
```

## 分け方をすべて列挙

```
(run* (l r) (appendo l r '(a b c)))
⇒ (((() (a b c)) ((a) (b c))
      ((a b) (c)) ((a b c) ())))
```

「実行方向」という概念が、そもそもない。

ふつうの言語には、これができない。

# これだけです

プログラム = 関係。 問いの向きは自由。

`run*` でクエリ

`==` と `fresh` で制約

`conde` で選択肢指定

(ベースとなる言語——たとえば Scheme——に加えて)

この「向き不問でクエリできる」力を医療知識に向けたものが、後半に登場します。

# いま面白い、PL 研究の最前線

## 代数的エフェクト

Koka · Eff · OCaml 5～

型付けは現在進行形の研究

## 量子計算向け言語

Q# · Silq

## 多段階プログラミング

Disciplined Multi-Stage

Programming

## 確率的プログラミング

Stan · PyMC

ベイズ統計 → 医療とも関連

## Rust

アフィン型 · 所有権追跡

産業界で採用されつつ

## 検証指向言語

F\* · Dafny · Lean

ほかにも：relational properties · dependent types · gradual typing · session types · separation logic · refinement types · staged compilation · …

PL の一分野：形式手法（Formal Methods, FM） — 形式仕様と証明でソフトウェアの性質を保証

# 発表者の専門 — DSL の設計・実装

DSL (Domain-Specific Language) : 特定の応用ドメインのための特化型言語



その言語を設計し、実装するのが DSL 屋の仕事。  
その言語でドメインロジックを記述するのはドメインエキスパート。

# 例：SQL 対「同じことをするプログラム」

## SQL

```
SELECT p.name, MAX(v.temp_c)
FROM patients p
JOIN vitals v
  ON v.patient_id = p.id
WHERE v.taken_at >=
      DATE('now', '-7 day')
GROUP BY p.id
HAVING MAX(v.temp_c) >= 38.0;
```

## 「同じこと」を手続きで

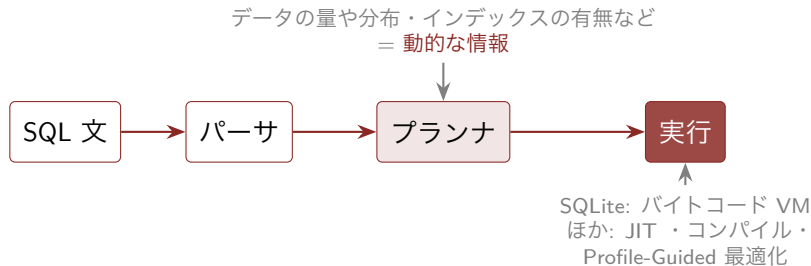
```
hot = {}
for v in read_all(vitals):
    if v.taken_at < week_ago:
        continue
    t = hot.get(v.patient_id)
    if t is None or v.temp_c > t:
        hot[v.patient_id] = v.temp_c
out = []
for p in read_all(patients):
    t = hot.get(p.id)
    if t is not None and t >= 38.0:
        out.append((p.name, t))
```

右は実行手順を固定してしまう。左は「何が欲しいか」だけを言う。

※同じクエリを計算する方法は何通りもある——右の手続きはその一つにすぎない



## (続) クエリエンジンが実行計画を動的に構築



同じクエリが、データベースの状況に応じて**その場で最適な手順**で実行される。  
「何が欲しいか」だけ簡潔に記載する言語設計が可能 — これも**言語的抽象の恩恵**の 1 つ

# で、医療との関係は？

## バイオインフォマティクス

ゲノム・オミクス解析

パイプラインだらけの世界

## 医療機器

組込み・計測制御

(LabVIEW、再登場)

## 医学・創薬研究

統計解析・ベイズ推論

Stan などが現役

## AI 開発

DeepMind 系：AlphaFold

タンパク質構造予測

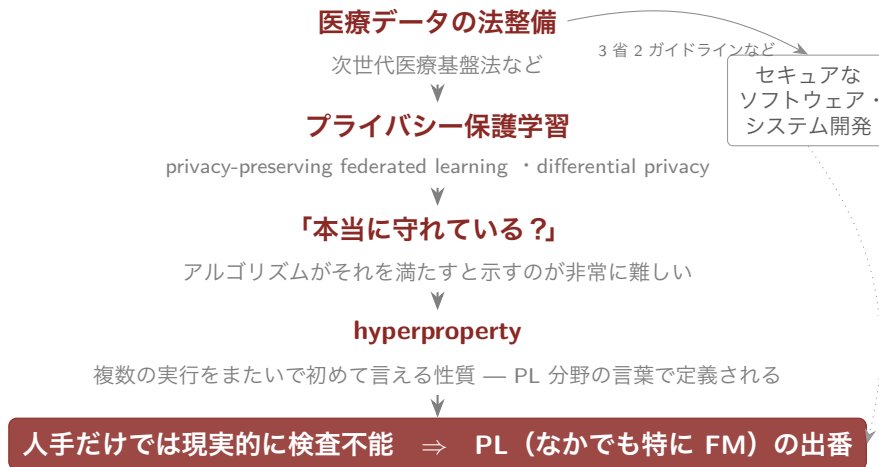
AlphaFold は 2024 年ノーベル化学賞 (Hassabis & Jumper ら)。  
計算がそのまま医学の道具になる時代。

どの軸でも、**ソフトウェア = 言語で書かれたもの**が主役。

他にも医療現場で使われている各種の情報システムも当然プログラミング言語で記述されている。

**大アリです。**

# プライバシー保護にも



今日はその中でも、  
PL が直接医療と関わった事例  
を見ていきます。

# ある PL 研究者



Matt Might

PL 研究者（プログラム静的解析）

University of Utah ・ CS 教員（約 10 年）

ブログ `matt.might.net` は CS/PL 界隈で有名

## 終わらない検査 — 「診断の旅」

Might 氏の長子 Bertrand には、乳児期から様々な症状があった  
しかしなかなか**診断がつかず**根治ができなかった

発達の遅れ

運動障害・けいれん発作

肝機能障害

検査、また検査    どの既知疾患にも合致しない

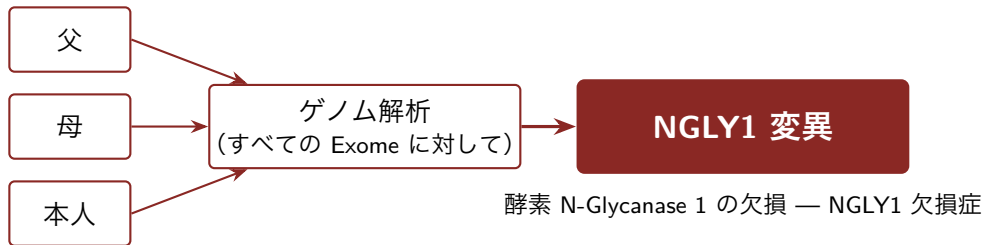
誕生 3 年



既知の代謝疾患・遺伝疾患を次々と除外 — それでも答えは出ない

# ゲノム解析

「診断の旅」に終止符を打ったのは、  
未発見な遺伝的障害を検知するパイロットプログラムへの参加



Duke 大学の研究チームによる解析で判明

長子 Bertrandさんは **Global Patient #0** (世界初の確定症例)

この診断によりやっと有効な治療が可能となり、病状が好転に向った

## 2012 年、ブログ記事公開に

### *Hunting down my son's killer*

Matt Might, 2012



[https://matt.might.net/articles/  
my-sons-killer/](https://matt.might.net/articles/my-sons-killer/)

[MATT.MIGHT.NET](https://matt.might.net) | [ARTICLE INDEX](#) | [@MATTMIGHT](#) | [RSS](#)

### Hunting down my son's killer

I found my son's killer.

It took three years.

But we did it.



*Not quite like this.*

I should clarify one point: my son is very much alive.

Yet, my wife Cristina and I have been found responsible for his death.

※Bertrand さんはその後残念ながら 2020 年に 12 歳で逝去



# 記事が患者を見つけた

記事は技術系コミュニティを超えて拡散し、  
世界中の家族が「うちの子も」と気づくきっかけとなり

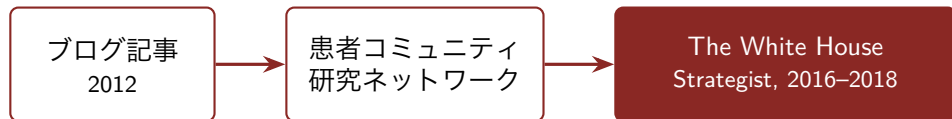
1 → +2 → +8 → +15 ...

のように記事の「Update:」欄の確定症例数が書き換えられ続けた

発見プロセス自体が論文に — *Genetics in Medicine* (患者家族との共著)

M. Might and M. Wilsey, "The shifting model in clinical diagnostics: how next-generation sequencing and families are altering the way rare diseases are discovered, studied, and treated," *Genetics in Medicine* 16(10):736–737, 2014. doi:10.1038/gim.2014.23

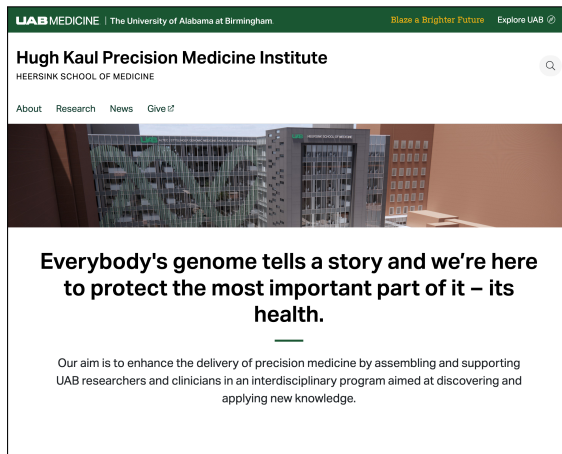
# ブログ記事から、国家戦略へ



オバマ政権の Precision Medicine Initiative を NIH らと推進

そして 2017 年、彼はある先端医療技術の研究所の所長になる —

# UAB Hugh Kaul Precision Medicine Institute



UAB PMI:  
アラバマ大学バーミンガム校  
Hugh Kaul  
精密医療研究所  
(Heersink School of Medicine 内)

[uab.edu/medicine/pmi](http://uab.edu/medicine/pmi)

# 精密医療 (Precision Medicine) の中枢に PL 研究者



**Matt Might**

PL 研究者 (Univ. of Utah、約 10 年)



Director, Hugh Kaul PMI (2017-)



**William E. Byrd**

PL 研究者 — 現役のまま UAB PMI に

“Scientist” として採用

→ 現在は Associate Director に昇進

(Scientist という肩書が医療機関に存在すること自体が珍しいのではないか)

# 精密医療 (Precision Medicine) の中枢に PL 研究者



**Matt Might**

PL 研究者 (Univ. of Utah、約 10 年)



Director, Hugh Kaul PMI (2017-)



**William E. Byrd**

PL 研究者 — 現役のまま UAB PMI に

**“Scientist”** として採用

→ 現在は Associate Director に昇進

(Scientist という肩書が医療機関に存在すること自体が珍しいのではないか)

# miniKanren × 医学知識 = mediKanren

さっき覚えたあの言語が、  
そのまま医療の道具になる

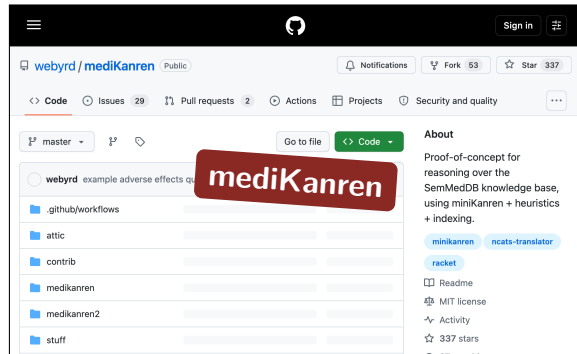
関連プログラミング言語 miniKanren

×

NIH NCATS Data Translator

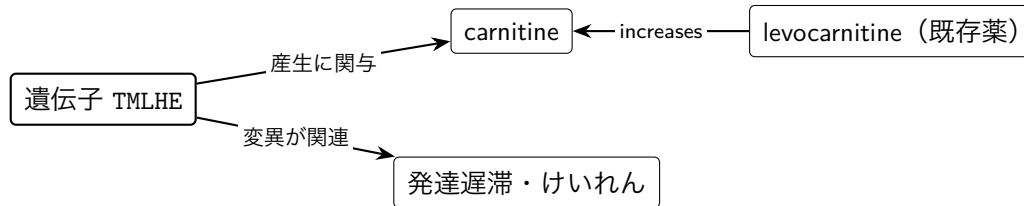
×

生物・医学ナレッジベース



github.com/webyrd/mediKanren (MIT license・研究用途)

# 生物医学ナレッジグラフ



ナレッジグラフ: 文献・データベース由来の「主語 + 述語 + 目的語」トリプルによる巨大なネット

SemMedDB

RTX-KG2

ROBOKOP

(Terminology は Biolink で統一)

# ナレッジグラフ → mediKanren

(増やす 'levocarnitine' 'carnitine')

※ナレッジグラフ上の辺は「関係」としてみなせる

Q: 「carnitine を増やす薬は?」

(run\* (薬)  
 (増やす 薬 'carnitine))

⇒ levocarnitine, ...

Q: 「この薬は何を増やす?」

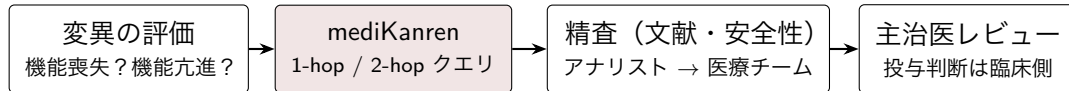
(run\* (何を)  
 (増やす 'levocarnitine 何を))

⇒ carnitine, ...

ナレッジグラフ上のパスを、どの向きクエリにも用いられる — まさに関係プログラミング



# UAB PMI での mediKanren 運用プロセスの 1 例



投薬候補は原則 FDA 承認済みの既存薬 — ドラッグ・リポジショニング

出典: A. Foksinska et al., "The precision medicine process for treating rare disease using the artificial intelligence tool mediKanren," *Frontiers in Artificial Intelligence* 5:910216, 2022. doi:10.3389/frai.2022.910216

## 成果（同研究が 2022 年時点において）

500+

希少疾患ケース  
当事者を支援

20+

既存薬から投与候補を  
同定できた症例

$> 1/2$

うち実際に投与へ  
進んだ割合（主治医判断）

出典: A. Foksinska et al., "The precision medicine process for treating rare disease using the artificial intelligence tool mediKanren,"  
*Frontiers in Artificial Intelligence* 5:910216, 2022. doi:10.3389/frai.2022.910216

# 具体例（同論文より）

## TMLHE 変異 × levocarnitine

毎日のけいれん発作・言語発達の遅れの小児例。mediKanren による 1-hop クエリが「levocarnitine *increases* carnitine」を提示。

家族の報告では運動機能が改善された

## RHOBTB2 変異 × celecoxib

けいれん・発達退行の小児 2 例。mediKanren による 2-hop クエリ（RHOBTB2 → E2F1 → その抑制薬）が celecoxib を提示。

2 例の主治医とも試験的投与に進むことを決定

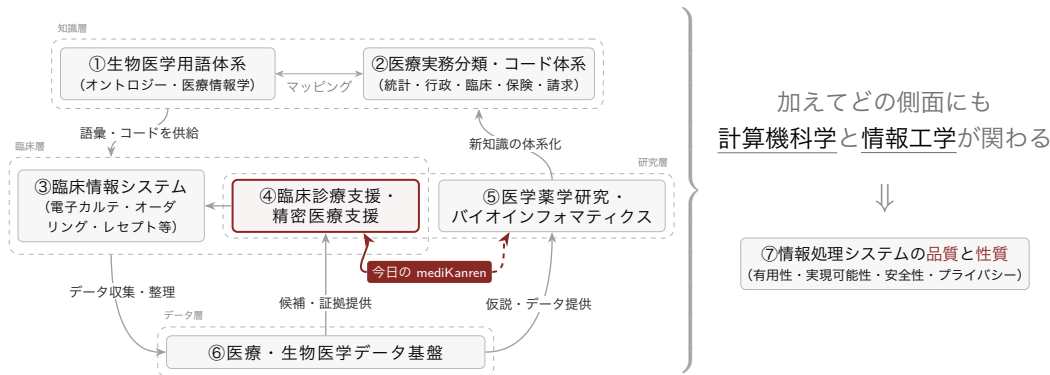
ナレッジグラフ上の「パス」が、治療仮説候補として浮上するようになる

出典: A. Foksinska et al., "The precision medicine process for treating rare disease using the artificial intelligence tool mediKanren," *Frontiers in Artificial Intelligence* 5:910216, 2022. doi:10.3389/frai.2022.910216



プログラミング言語論が医療に直接貢献した、現在進行形の 1 例

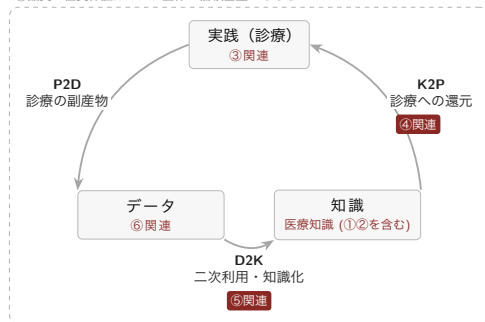
# 発表者の関心：応用ドメインとしての医療・医学（特に情報科学と関連する部分）



①～⑦のいずれも良いソフトウェア／適切な情報科学的手法と関連し、  
自然にプログラミング言語論とも関連性を持つ

# 偶然の一致：Learning Health System (=「学習する医療システム」)

⑦品質と性質保証はループ全体の信頼基盤ともなる



※LHS は医療知識・医療現場・組織の文化・インセンティブなどを含めて扱う概念; 対して前スライドの図はその情報学的側面のみを扱うもの  
 ※LHSにおいて①② (用語・コード体系) は知識化の成果物ではなく基盤側に分類されるもの

## Learning Health System (IOM, 2007-)

新しい知識が、診療の副産物として  
絶えず生まれ続けるシステム

*"new knowledge captured as an integral  
by-product of the care experience"*

発表者の関心事項を整理していたら、  
LHS との類似が AI に指摘された

出典: IOM, *Best Care at Lower Cost: The Path to Continuously Learning Health Care in America*, National Academies Press, 2013.  
 doi:10.17226/13444 / C. P. Friedman et al., *Yearb. Med. Inform.* 26(1):16-23, 2017. doi:10.15265/IY-2017-004

# ご清聴ありがとうございました

Haochen Kotoi-Xie  
[github.com/haochenx](https://github.com/haochenx)

## 改めて、おすすめ記事

[miniKanren.org](#) / [articles/news](#) / [@haochenx](#) / [xie](#)

### Hunting down my son's killer

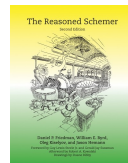
I found my son's killer  
It took three years  
But we did it.



Not quite like this.  
I should clarify one point: my son is very much alive.  
No, my wife Christina and I have been found responsible for his death.

*Hunting down my son's killer* — Matt Might  
[matt.might.net/articles/my-sons-killer/](https://matt.might.net/articles/my-sons-killer/)

## miniKanren に興味が湧いたら



*The Reasoned Schemer*, 2nd ed. (MIT Press)  
[mitpress.mit.edu/9780262535519/the-reasoned-schemer/](https://mitpress.mit.edu/9780262535519/the-reasoned-schemer/)

※本資料の調査・作成には LLM を活用しています

# 目次

- 1 プログラミング言語の世界
- 2 miniKanren 入門
- 3 プログラミング言語、再び
- 4 Matt Might と Bertrand
- 5 mediKanren と UAB PMI
- 6 発表者の関心



# 図版クレジット

## Wikimedia Commons (オープンライセンス)

|                     |                                                            |               |
|---------------------|------------------------------------------------------------|---------------|
| LabVIEW ブロックダイアグラム  | File:LabVIEW Block diagram.JPG                             | Public Domain |
| Scratch 3.0 画面      | File:Scratch 3.0 Screenshot Green Flag Clicked Example.png | CC BY-SA 4.0  |
| LEGO Mindstorms RCX | File:LegoMindstormsRCX.jpg                                 | CC BY-SA 3.0  |
| LEGO Roverbot (実機)  | File:Lego Roverbot.JPG (Legoktm 氏)                         | CC BY 2.5     |
| ROBOLAB 画面          | File:Robolabscreenshot.png                                 | Public Domain |

## スクリーンショット (発表者による取得)

|                                        |                                                   |
|----------------------------------------|---------------------------------------------------|
| 記事 <i>Hunting down my son's killer</i> | matt.might.net/articles/my-sons-killer/           |
| UAB Hugh Kaul PMI                      | uab.edu/medicine/pmi                              |
| mediKanren リポジトリ                       | github.com/webyrd/mediKanren (リポジトリは MIT License) |

## 写真・カバー (権利は各権利者に帰属)

|                                       |                               |
|---------------------------------------|-------------------------------|
| William E. Byrd 氏近影                   | webyrd.net 掲載の本人写真            |
| Matt Might 氏近影                        | UAB 公式ページ (uab.edu) 掲載のポートレート |
| <i>The Reasoned Schemer</i> , 2nd ed. | カバー (MIT Press)               |

※上記以外の図・コード例はすべて発表者によって作成されたものです。

## 予備スライド一覧

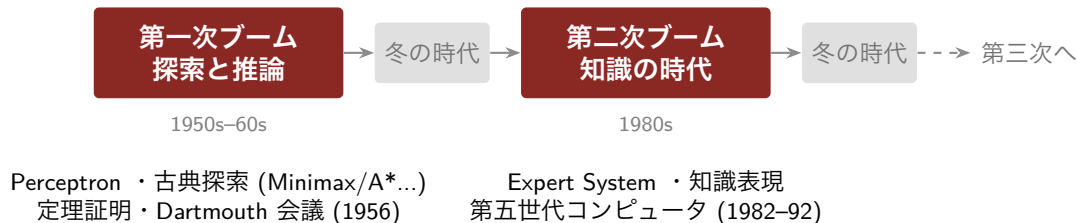
AI ブームの過去と現在 (1) : 第一次・第二次

AI ブームの過去と現在 (2) : 第三次～

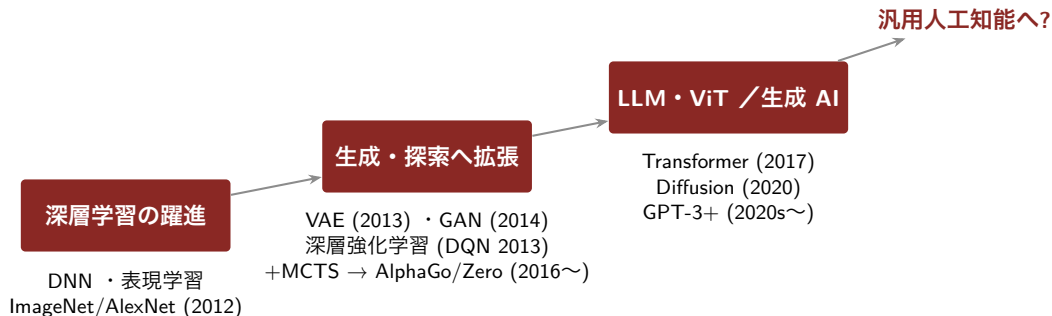
記号主義 vs コネクショニズム

3 省 2 ガイドラインとは

# AI ブームの過去と現在 (1)：第一次・第二次



## AI ブームの過去と現在 (2) : 第三次～



※生成 AI が汎用人工知能への唯一の突破口とは限らない—他のアプローチも並走している

◀ 一覧へ

# 記号主義 vs コネクショニズム

## 記号主義 (Symbolic AI)

論理・知識表現・明示的な推論  
Expert System ・ Prolog ・ 定理証明...

VS

## コネクショニズム

学習・分散表現・確率論的な振舞  
Perceptron → DNN → LLM...

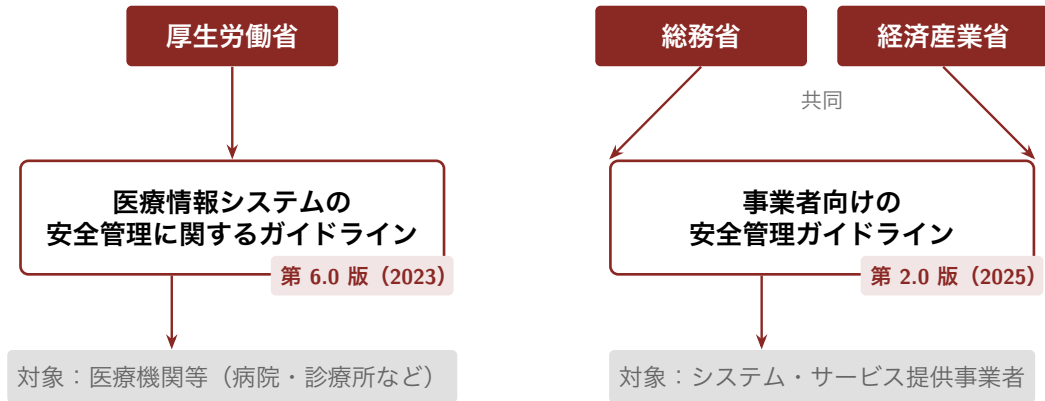
両者が統合されて動くシステムが必要

ブームのたびに交互に主役を演じてきた — 対立ではなく有機的な統合こそが正解

◀ 一覧へ

## 3 省 2 ガイドラインとは

3 つの省が定める 2 本の医療情報セキュリティガイドラインの総称



◀ 一覧へ

注：医療機関等向けガイドラインは 2026 年 6 月に第 7.0 版へ改定されている